

“Año de la recuperación y consolidación de la economía peruana”

Universidad San Ignacio de Loyola



Título

Desarrollo de un DSL para a automatización de operaciones en e-commerce de moda

Facultad de Ingeniería

Curso

Compiladores

Integrantes

Barrantes Lopez, Sebastian Enrique	2210781
Quispe Bartolo, Carlos Martin	2312718
Vasquez Vasquez Diego Alonso	2220670

Profesor

Dioses Zarate, Javier Antonio

Lima, Perú

2025 – 02

Tabla de Contenidos

Tabla de Contenidos	2
1. Introducción	3
2. Descripción del caso de estudio	4
2.1. Análisis de Ventas y Generación de Reportes	4
2.2. Gestión de Inventario y Reposición Automática	4
2.3. Identificación de "Slow Movers"	5
2.4. Automatización de Promociones y Descuentos	5
3. Objetivos	6
3.1. Objetivo General	6
3.2. Objetivos Específicos	6
4. Desarrollo	7
4.1. Identificación de Requerimientos	7
4.1.1. Requerimientos Funcionales	7
4.1.2. Requerimientos no Funcionales	7
4.2. Solución del Caso	8
4.2.1. Diseño del Lenguaje	8
4.2.2. Implementación del Análisis Léxico	13
4.2.3. Implementación del Análisis Sintáctico	15
4.2.4. Implementación del Análisis Semántico	18
4.2.5. Traducción Dirigido por la Sintaxis	21
4.2.6. Aplicación Práctica	22
Conclusiones y Referencias	23
Referencias	24

1. Introducción

En la última década, el comercio electrónico (e-commerce) ha trascendido su rol de canal de venta alternativo para consolidarse como el motor principal del retail global. Impulsado por la conectividad ubicua y cambios en el comportamiento del consumidor, el mercado digital experimenta un crecimiento exponencial. Dentro de este vasto ecosistema, el sector de la moda y vestimenta (fashion e-commerce) emerge como uno de los segmentos más grandes y, a su vez, más operativamente complejos (Radial, 2024).

A diferencia de otros rubros, el e-commerce de moda enfrenta desafíos únicos que intensifican la necesidad de una gestión de procesos ágil y automatizada. Como lo menciona Debrouwere (2025), la naturaleza misma del producto introduce una complejidad logística exponencial: cada artículo se multiplica por un alto número de SKUs (Unidades de Mantenimiento de Stock) que incluyen tallas, colores y colecciones. Esta granularidad del inventario debe gestionarse en un entorno de alta volatilidad, marcado por la estacionalidad, las micro-tendencias del fast fashion y una presión constante por la renovación del catálogo.

Esta confluencia de inventarios complejos y reglas de negocio dinámicas crea una brecha significativa entre las necesidades operativas y las herramientas técnicas disponibles. Por esta razón, proponemos el diseño de un lenguaje de dominio específico (DSL). Como se explica en la página de JetBrains (s.f.), estos lenguajes se optimizan específicamente para usar los conceptos y reglas propios del dominio de una empresa. El beneficio de un DSL más evidente es la eficiencia: en lugar de que los desarrolladores realicen "el trabajo pesado manualmente", el DSL permite automatizar tareas repetitivas. Más importante aún, un DSL permite "separar lo esencial de la complejidad incidental", logrando que los pensamientos se vuelvan más claros y que el código sea más mantenible. Al estar diseñado en estrecha coordinación con los expertos del campo (gerentes de inventario, analistas de marketing, etc.), este lenguaje permite una excelente integración entre los expertos en TI y los expertos del dominio, facilitando que personal no programador pueda definir y ejecutar reglas de negocio complejas, mejorando así la agilidad y la calidad del producto final.

2. Descripción del caso de estudio

En el sector del e-commerce de moda, **NOVAÉ** se posiciona como una empresa innovadora dedicada a ofrecer ropa contemporánea a través de canales digitales. En su operación, la coordinación entre las distintas áreas del negocio —como ventas, inventario y promociones— resulta crítica para sostener la competitividad y garantizar una experiencia de compra fluida para sus clientes. Dado que las tendencias en el mercado de la moda cambian rápidamente y la demanda puede variar en cuestión de horas, **NOVAÉ** basa sus decisiones en datos precisos y actualizados que permiten anticipar comportamientos del consumidor y optimizar la gestión de sus recursos. A continuación, se describen los principales procesos de negocio que conforman la operación diaria de **NOVAÉ**, destacando su relevancia y los desafíos que implican para mantener la eficiencia y la satisfacción del cliente.

2.1. Análisis de Ventas y Generación de Reportes

El área de ventas constituye el núcleo de toda operación de comercio electrónico. En una tienda de ropa online, los reportes de ventas no solo reflejan el rendimiento económico, sino que también proporcionan información clave sobre las preferencias de los consumidores, el comportamiento de compra y la efectividad de las campañas de marketing. Con esto en mente, los reportes mensuales facilitan el análisis estratégico, permitiendo comparar resultados con periodos anteriores y ajustar proyecciones de ingresos. Estos reportes automatizados no solo ahorran tiempo operativo, sino que reducen el riesgo de errores humanos y permite que los equipos se concentren en la toma de decisiones basada en datos.

2.2. Gestión de Inventario y Reposición Automática

El control del inventario es uno de los mayores retos debido a la diversidad de tallas, colores y modelos. Una gestión ineficiente puede traducirse en pérdidas significativas: desde

ventas no concretadas por falta de stock hasta exceso de inventario que obliga a liquidar productos con descuentos agresivos.

Por esta razón se plantea implementar una reposición automática para introducir un componente de inteligencia al proceso: el sistema puede prever cuándo un artículo alcanzará un nivel mínimo de existencias y generar una orden de reabastecimiento de manera automática. Así garantizar que los productos más demandados estén siempre disponibles, mejorando la satisfacción del cliente y optimizando el flujo de caja.

2.3. Identificación de "Slow Movers"

Dentro del mismo inventario, otro aspecto crucial es la identificación de los llamados *slow movers*, estos representan de los mayores drenajes financieros para un retailer de moda. Como explica Jenkins (s.f.) en su análisis sobre gestión de inventario, este tipo de productos no solo incrementa los costos de mantenimiento (almacenamiento, seguros, personal), sino que, de forma crítica, inmoviliza el capital de trabajo. En el volátil rubro de la moda, donde las temporadas y tendencias garantizan la obsolescencia, un *slow mover* no gestionado se convierte rápidamente en una pérdida total.

El análisis de los *slow movers* no solo impacta en la gestión del inventario, sino que también aporta información valiosa a las áreas de diseño y marketing, ayudando a comprender mejor las preferencias del cliente y las tendencias del mercado.

2.4. Automatización de Promociones y Descuentos

El área de promociones cumple un rol fundamental en la fidelización del cliente y en la dinamización de las ventas. Sin embargo, la gestión manual de descuentos y campañas puede resultar compleja y propensa a errores, especialmente cuando se manejan múltiples categorías, temporadas o niveles de descuento.

La automatización de descuentos permite que el sistema aplique reglas previamente definidas según condiciones específicas: por ejemplo, aplicar un 20% de descuento a productos que lleven más de 60 días en stock, o activar ofertas relámpago durante fechas especiales como el *Black Friday* o *Cyber Wow*. Por lo tanto, mejorar la eficiencia operativa garantiza coherencia en la aplicación de las promociones y permite responder con agilidad ante cambios en el comportamiento del mercado.

3. Objetivos

3.1. Objetivo General

El objetivo general es desarrollar parte de un compilador que sea capaz de procesar un lenguaje para la automatización de operaciones en el sector de ropa e-commerce.

3.2. Objetivos Específicos

- Desarrollar el análisis léxico, sintáctico y semántico de nuestro lenguaje propuesto.
- Definir y formalizar la gramática del DSL, incluyendo construcciones específicas para reglas de negocio en e-commerce de moda, como queries de inventario y aplicación de descuentos.
- Implementar la fase de generación de código que permita ejecutar comandos para automatizar procesos como la reposición de stock.
- Realizar pruebas y validación del compilador utilizando casos de estudio reales del sector.

4. Desarrollo

4.1. Identificación de Requerimientos

Para el diseño e implementación orientado a la automatización de operaciones en e-commerce de moda, se identificaron los requerimientos a partir del análisis del caso de estudio. Estos se dividen en requerimientos funcionales y no funcionales. A continuación se presentan estos requerimientos:

4.1.1. Requerimientos Funcionales

Estos definen las operaciones que se debe soportar para automatizar tareas repetitivas y reglas de negocio.

- RF1: Generación de reportes de ventas - El lenguaje debe permitir definir comandos para analizar datos de ventas, generar reportes mensuales automáticos que incluyan métricas como ingresos, reduciendo errores humanos y facilitando comparaciones con periodos anteriores.
- RF2: Gestión y reposición automática de inventario - Debe incluir construcciones para monitorear niveles de stock, prever umbrales mínimos y generar órdenes de reabastecimiento automáticas para productos de alta demanda, optimizando el flujo de caja y evitando pérdidas por falta o exceso de stock.
- RF3: Identificación de slow movers - Debe soportar queries para detectar productos de baja rotación basados en criterios como días en stock, ventas mínimas y costos de mantenimiento, proporcionando insights para áreas de diseño y marketing, y facilitando decisiones como descuentos o eliminación de ítems obsoletos.
- RF4: Automatización de promociones y descuentos - Permitir definir reglas condicionales para aplicar descuentos automáticos, asegurando coherencia y agilidad en respuesta a cambios de mercado.
- RF5: Soporte para estructuras de control - Incluir elementos como variables, constantes, arreglos, condicionales, bucles y funciones para manejar lógica compleja en reglas de negocio.
- RF6: Integración con datos externos - El compilador debe permitir la interpretación de comandos que interactúen con bases de datos simuladas o archivos de entrada para procesar información real de e-commerce.

4.1.2. Requerimientos no Funcionales

Estos aseguran la calidad y viabilidad del sistema.

- RNF1: Usabilidad - Debe ser intuitivo para usuarios no programadores, utilizando sintaxis cercana al dominio para facilitar la definición de reglas sin conocimiento profundo de programación.
- RNF2: Eficiencia - El compilador debe procesar programas en tiempo razonable, menos de un segundo por comando simple, optimizando el análisis léxico, sintáctico y semántico para manejar volúmenes de datos típicos en e-commerce.
- RNF3: Mantenibilidad - La gramática y el código del compilador deben ser modulares, permitiendo extensiones futuras para nuevos procesos de negocio sin reescrituras mayores.
- RNF4: Seguridad - Asegurar que el lenguaje no permita operaciones que comprometan datos sensibles, como validaciones semánticas para evitar inyecciones o accesos no autorizados en simulaciones.

4.2. Solución del Caso

Para abordar los requerimientos identificados en la sección anterior, se propone el desarrollo de un subconjunto de un compilador que procese un lenguaje de dominio específico denominado FashionOpsDSL. Este lenguaje está orientado a la automatización de operaciones en el e-commerce de moda, permitiendo a usuarios no técnicos definir y ejecutar reglas de negocio de manera intuitiva.

4.2.1. Diseño del Lenguaje

El diseño de FashionOpsDSL se basa en principios de DSLs para dominios de negocio, inspirados en enfoques como los descritos en literatura sobre lenguajes para supply chain. A continuación, se detalla la gramática en sus componentes léxica, sintáctica y semántica.

Primero iniciamos con la definiciones de los tokens básicos mediante expresiones regulares para posteriormente implementarlos dentro de un analizador léxico.

Token	Patrón (Expresión Regular)	Descripción
IDENTIFICADOR	[a-zA-Z_][a-zA-Z0-9_]*	camisa_azul, stock_total

NUMERO	[0-9]+(\.[0-9]+)?	10, 20.5
CADENA	"[^"]*"	"M", "azul"
PALABRA_RESERVADA	REPORTAR VER_STOCK REPONER IDENTIFICAR_LENTOS APLICAR_DESCUENTO SI ENTONCES SINO FIN PARA EN HAZ VARIABLE CONSTANTE ARREGLO DIARIO MENSUAL FILTRAR ES DIAS VENTAS	REPORTAR, SI
OPERADOR_RELACIONAL	'== !=	>
OPERADOR_LOGICO	Y O NO	Y, O
OPERADOR_ARITMETICO	`+`	-
ASIGNACION	=	=
SIMBOLO	[;,\[\]\{\}\%]	;; %
COMENTARIO	//.*	// nota
ESPACIO	[\t\n\r]+	(Ignorado)

Dentro de estos tokens se consideran palabras reservadas que permiten automatizar reglas de negocio. A continuación se explican estas palabras y su función:

- REPORTAR: Genera reportes automáticos de ventas (diarios/mensuales) con filtros opcionales.
- VER_STOCK: Consulta el nivel actual de inventario de un SKU específico.
- REPONER: Ordena reabastecimiento automático de un producto cuando el stock es bajo.
- IDENTIFICAR_LENTOS: Identifica productos con baja rotación según días en stock y ventas mínimas.

- APLICAR_DESCUENTO: Aplica descuentos porcentuales a productos, opcionalmente bajo condiciones.
- SI|ENTONCES|SINO|FIN: Estructura condicional: ejecuta bloques de código si una condición es verdadera.
- PARA|EN|HAZ: Bucle que itera sobre un arreglo para aplicar reglas.
- VARIABLE: Declara variables mutables.
- CONSTANTE: Declara constantes.
- ARREGLO: Crea arreglos de valores homogéneos para procesar listas de SKUs, ventas, etc.
- DIARIO|MENSUAL|FILTRAR|ES|DIAS|VENTAS: Filtros

Una vez definida los tokens, parte léxica del lenguaje, se define la sintaxis de nuestro DSL para FashionOpsDSL. La gramática presentada es libre de contexto y se usará para construir el analizador sintáctico descendente (LL(1)), también la gramática es no ambigua, y sin recursión izquierda. A continuación, se presentan las reglas de producción de la gramática:

No-terminal	Producción
<programa>	<sentencia>*
<sentencia>	<declaracion> ";" <asignacion> ";" <reporte> ";" <ver_stock> ";" <reponer> ";" <identificar_lentos> ";" <aplicar_descuento> ";" <si> <para>
<declaracion>	"VARIABLE" <id> "=" <expresion> "CONSTANTE" <id> "=" <expresion> "ARREGLO" <id> "=" "[" <lista_expresiones> "]"
<asignacion>	<id> "=" <expresion>
<reporte>	"REPORTAR" "VENTAS" ("DIARIO" "MENSUAL") ["FILTRAR" <condicion>]
<ver_stock>	"VER_STOCK" <id> "ES" <expresion>
<reponer>	"REPONER" <id> <expresion>
<identificar_lentos>	"IDENTIFICAR_LENTOS" "DIAS" ">" <numero> ["VENTAS" "<" <numero>]

<aplicar_descuento>	"APLICAR_DESCUENTO" <id> <numero> "%" ["SI" <condicion>]
<si>	"SI" <condicion> "ENTONCES" <sentencia>* ["SINO" <sentencia>*] "FIN"
<para>	"PARA" <id> "EN" <id> "HAZ" <sentencia>* "FIN"
<expresion>	<termino> (("+" "-") <termino>)*
<termino>	<factor> (("*" "/") <factor>)*
<factor>	<numero> <cadena> <id> <acceso_arreglo> "(" <expresion> ")"
<acceso_arreglo>	<id> "[" <expresion> "]"
<lista_expresiones>	<expresion> ("," <expresion>)* ε (ε = cadena vacía)
<condicion>	<expresion> <op_relacional> <expresion> "NO" <condicion> <condicion> ("Y" "O") <condicion> "(" <condicion> ")"
<op_relacional>	"==" "!=" ">" ">=" "<" "<="
<id>	IDENTIFICADOR
<numero>	NUMERO
<cadena>	CADENA

A continuación, se definen reglas semánticas del lenguaje para garantizar el sentido y significado de las palabras aceptadas para cumplir con las reglas del negocio.

Categoría	Regla	Descripción	Ejemplo de Error Detectado
Tipado	Inferencia de tipos	Las variables (VARIABLE) y constantes (CONSTANTE) infieren su tipo a partir de la expresión asignada. Los arreglos (ARREGLO) deben contener elementos del mismo tipo.	VARIABLE x = "camisa"; x = 10; → <i>Error: asignación de entero a variable de tipo cadena</i>

	Homogeneidad en arreglos	Todos los elementos de un ARREGLO deben ser del mismo tipo.	ARREGLO items = ["M", 10]; → <i>Error: arreglo con tipos mixtos</i>
Alcance	Variables locales	Las variables declaradas dentro de bloques SI o PARA solo existen en ese bloque.	SI stock < 10 ENTONCES VARIABLE temp = 5; FIN; temp = 3; → <i>Error: variable 'temp' no definida</i>
	Inmutabilidad de constantes	Una CONSTANTE no puede reasignarse después de su declaración.	CONSTANTE min = 10; min = 15; → <i>Error: reasignación de constante</i>
Validación de Dominio	Cantidades positivas en reponer	En REPONER, la cantidad debe ser mayor que 0.	REPONER "camisa" -5; → <i>Error: cantidad de reabastecimiento debe ser positiva</i>
	Porcentajes válidos	En APLICAR_DESCUENTO, el porcentaje debe estar entre 0 y 100.	APLICAR_DESCUENT O "pantalon" 150%; → <i>Error: porcentaje fuera de rango (0–100)</i>
Existencia de SKUs	Productos reales	Cualquier referencia a un producto (en VER_STOCK, REPONER, etc.) debe existir en el catálogo de entrada (simulado vía CSV o base de datos).	VER_STOCK "zapatilla_X" ES 5; → <i>Error: SKU 'zapatilla_X' no encontrado en inventario</i>
Condiciones y Tipos	Compatibilidad relacional	Los operadores de comparación (==, <, >, etc.) solo se permiten entre operandos del mismo tipo.	"M" > 10 → <i>Error: no se puede comparar cadena con número</i>
Acceso a Arreglos	Índices válidos	El índice en arr[i] debe ser un entero y estar dentro de los límites del arreglo.	ARREGLO sizes = ["S", "M"]; sizes[5]; → <i>Error: índice fuera de rango</i>

Efectos Laterales	Modificación de estado	Comandos como REPONER actualizan el estado simulado del inventario. El analizador verifica que no haya efectos colaterales no permitidos.	REPONER "camisa" 0; → <i>Advertencia:</i> <i>reabastecimiento sin cantidad útil</i>
--------------------------	------------------------	---	---

El analizador semántico recorre el Árbol Sintáctico Abstracto generado en la fase sintáctica, verifica tipos, alcances y restricciones del dominio, y anota el árbol con información de tipos. Si se detecta un error, se genera un mensaje claro para el usuario.

4.2.2. Implementación del Análisis Léxico

Para la implementación del análisis léxico se utilizó la herramienta Flex para poder definir los tokens mediante las expresiones regulares que definimos previamente. A continuación se muestra el archivo flex utilizado.

```
%{
#include "fashion-ops-bison.tab.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
%}

%option yylineno
%option noyywrap

%%

[ \t\r]+      ; /* ignorar espacios y tabuladores */
\n           { yylineno++; }
"//".*       ; /* comentarios de una línea */

"REPORTAR"    { return REPORTAR; }
"VENTAS"      { return VENTAS; }
"DIARIO"      { return DIARIO; }
"MENSUAL"     { return MENSUAL; }
"FILTRAR"     { return FILTRAR; }

"VER_STOCK"   { return VER_STOCK; }
"REPONER"     { return REPONER; }
"IDENTIFICAR_LENTOS" { return IDENTIFICAR_LENTOS; }
"APLICAR_DESCUENTO" { return APLICAR_DESCUENTO; }

"SI"          { return SI; }
"ENTONCES"    { return ENTONCES; }
"SINO"        { return SINO; }
```

```

"FIN"                { return FIN; }

"PARA"               { return PARA; }
"EN"                 { return EN; }
"HAZ"                { return HAZ; }

"VARIABLE"           { return VARIABLE; }
"CONSTANTE"          { return CONSTANTE; }
"ARREGLO"            { return ARREGLO; }

"ES"                 { return ES; }
"DIAS"               { return DIAS; }

"%"                  { return PORCENTAJE; }

"Y"                  { return Y; }
"O"                  { return O; }
"NO"                 { return NO; }

"=="                 { return IGUAL; }
"!="                 { return DIFERENTE; }
">="                 { return MAYOR_IGUAL; }
"<="                 { return MENOR_IGUAL; }
">"                 { return MAYOR; }
"<"                 { return MENOR; }

"+"                 { return MAS; }
"-"                 { return MENOS; }
"*"                 { return POR; }
"/"                 { return DIV; }

"="                  { return ASIGNAR; }
";"                  { return PUNTO_COMA; }
","                  { return COMA; }
"("                  { return PARENTESIS_IZQ; }
")"                  { return PARENTESIS_DER; }
"["                  { return CORCHETE_IZQ; }
"]"                  { return CORCHETE_DER; }

[0-9]+(\\.[0-9]+)?   { yylval.valor_real = atof(yytext); return NUMERO; }

\\"[^"]*"\\          { yylval.cadena = strdup(yytext); return CADENA; }

[a-zA-Z_][a-zA-Z0-9_]* { yylval.cadena = strdup(yytext); return IDENTIFICADOR; }

.                    { fprintf(stderr, "Error léxico en línea %d: carácter inesperado '%s\\n",
yylineno, yytext); exit(1); }

%%

```

```

int main() {
    yylex()
    return 0;
}

int yywrap() {
    return 1;
}

```

Posteriormente, se utilizó el comando flex para poder compilar el analizador léxico y generar el código en C.

4.2.3. Implementación del Análisis Sintáctico

Para la implementación del análisis sintáctico se utilizó la herramienta Bison para utilizar los tokens identificados y definir las reglas de producción de la gramática, de esta forma especificando la parte sintáctica del lenguaje. A continuación se muestran las reglas de producción creadas, primero se implementó las reglas sin su funcionamiento.

```

programa:
    /* vacío */
    | programa sentencia
    | programa si
    | programa para
    ;

sentencia:
    declaracion PUNTO_COMA
    | asignacion PUNTO_COMA
    | reporte PUNTO_COMA
    | ver_stock PUNTO_COMA
    | reponer PUNTO_COMA
    | identificar_lentos PUNTO_COMA
    | aplicar_descuento PUNTO_COMA
    ;

declaracion:
    VARIABLE IDENTIFICADOR ASIGNAR expresion      { printf("Declaración
VARIABLE: %s\n", $2); free($2); }
    | CONSTANTE IDENTIFICADOR ASIGNAR expresion    { printf("Declaración
CONSTANTE: %s\n", $2); free($2); }
    | ARREGLO IDENTIFICADOR ASIGNAR CORCHETE_IZQ lista_expresiones
CORCHETE_DER
    { printf("Declaración ARREGLO: %s = [...]n", $2); free($2); }
    ;

```

asignacion:

```
IDENTIFICADOR ASIGNAR expresion      { printf("Asignación: %s = ...\n",
$1); free($1); }
;
```

reporte:

```
REPORTAR VENTAS DIARIO                { printf("Reporte de ventas
DIARIO\n"); }
| REPORTAR VENTAS MENSUAL              { printf("Reporte de ventas
MENSUAL\n"); }
| REPORTAR VENTAS DIARIO FILTRAR condicion { printf("Reporte de ventas
DIARIO con filtro\n"); }
| REPORTAR VENTAS MENSUAL FILTRAR condicion { printf("Reporte de
ventas MENSUAL con filtro\n"); }
;
```

ver_stock:

```
VER_STOCK IDENTIFICADOR ES expresion  { printf("Consulta stock: %s =
...\n", $2); free($2); }
;
```

reponer:

```
REPONER IDENTIFICADOR expresion      { printf("Reponer producto %s
con cantidad ...\n", $2); free($2); }
;
```

identificar_lentos:

```
IDENTIFICAR_LENTOS DIAS MAYOR NUMERO {
printf("Identificar lentos con más de %.0f días\n", $4); }
| IDENTIFICAR_LENTOS DIAS MAYOR NUMERO VENTAS MENOR NUMERO
{ printf("Identificar lentos > %.0f días y ventas < %.0f\n", $4, $7); }
;
```

aplicar_descuento:

```
APLICAR_DESCUENTO IDENTIFICADOR NUMERO PORCENTAJE
{ printf("Aplicar %.0f%% descuento a %s\n", $3, $2); free($2); }
| APLICAR_DESCUENTO IDENTIFICADOR NUMERO PORCENTAJE SI condicion
{ printf("Aplicar %.0f%% descuento condicional a %s\n", $3, $2); free($2); }
;
```

si:

```
SI condicion ENTONCES programa FIN
{ printf("Estructura SI-ENTONCES-FIN\n"); }
| SI condicion ENTONCES programa SINO programa FIN
{ printf("Estructura SI-ENTONCES-SINO-FIN\n"); }
;
```

para:

PARA IDENTIFICADOR EN IDENTIFICADOR HAZ programa FIN

```
{ printf("Bucle PARA %s EN %s HAZ ... FIN\n", $2, $4); free($2); free($4); }
```

;

expresion:

```
expresion MAS termino    { printf(" + "); }
```

```
| expresion MENOS termino { printf(" - "); }
```

```
| termino
```

;

termino:

```
termino POR factor      { printf(" * "); }
```

```
| termino DIV factor    { printf(" / "); }
```

```
| factor
```

;

factor:

```
NUMERO                  { printf("%.2f ", $1); }
```

```
| CADENA                 { printf("%s ", $1); free($1); }
```

```
| IDENTIFICADOR          { printf("%s ", $1); free($1); }
```

```
| acceso_arreglo
```

```
| PARENTESIS_IZQ expresion PARENTESIS_DER
```

;

acceso_arreglo:

```
IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER
```

```
{ printf("Acceso a arreglo: %s[...]\n", $1); free($1); }
```

;

lista_expresiones:

```
/* vacío */
```

```
| expresion
```

```
| lista_expresiones COMA expresion
```

;

condicion:

```
expresion IGUAL expresion { printf(" == "); }
```

```
| expresion DIFERENTE expresion { printf(" != "); }
```

```
| expresion MAYOR expresion { printf(" > "); }
```

```
| expresion MAYOR_IGUAL expresion { printf(" >= "); }
```

```
| expresion MENOR expresion { printf(" < "); }
```

```
| expresion MENOR_IGUAL expresion { printf(" <= "); }
```

```
| NO condicion { printf("NO "); }
```

```
| condicion Y condicion { printf(" Y "); }
```

```
| condicion O condicion { printf(" O "); }
```

```
| PARENTESIS_IZQ condicion PARENTESIS_DER
```

;

4.2.4. Implementación del Análisis Semántico

Para la implementación del análisis semántico se utilizó la herramienta Bison, integrando acciones semánticas en las reglas de producción definidas previamente en la sección sintáctica. Esto permite no solo validar la estructura del programa, sino también verificar reglas lógicas como tipos de datos, alcances de variables, inmutabilidad de constantes y restricciones específicas del dominio.

Se implementó una tabla de símbolos simple utilizando una lista enlazada para manejar declaraciones y referencias a variables, constantes y arreglos. A continuación se muestran las estructuras clave y las reglas de producción con sus acciones semánticas.

```
// Enumeración para tipos
typedef enum { T_UNKNOWN, T_NUMBER, T_STRING, T_ARRAY } Type;

// Estructura para expresiones (para verificación de tipos)
typedef struct {
    Type type;
    double num_val;
    char *str_val;
} Expr;

// Estructura para símbolos
typedef struct Symbol {
    char *name;
    Type type;
    int is_const;
    int scope;
    Type array_elem_type; // Para arreglos, el tipo de elementos
    struct Symbol *next;
} Symbol;

// Tabla de símbolos (lista enlazada simple por simplicidad, puede ser hash más adelante)
Symbol *sym_table = NULL;
int current_scope = 0;
```

Además se implementaron una serie de funciones para realizar el análisis semántico del lenguaje.

```
// Funciones para tabla de símbolos
Symbol* lookup(char *name) {
    Symbol *p = sym_table;
    int max_scope = -1;
    Symbol *found = NULL;
    while (p != NULL) {
        if (strcmp(p->name, name) == 0 && p->scope <= current_scope) {
```

```

        if (p->scope > max_scope) {
            max_scope = p->scope;
            found = p;
        }
    }
    p = p->next;
}
return found;
}

void insert(char *name, Type type, int is_const, Type array_elem_type) {
    if (lookup(name) != NULL) {
        yyerror("Error semantico: Variable ya declarada");
        exit(1);
    }
    Symbol *s = (Symbol*)malloc(sizeof(Symbol));
    s->name = strdup(name);
    s->type = type;
    s->is_const = is_const;
    s->scope = current_scope;
    s->array_elem_type = array_elem_type;
    s->next = sym_table;
    sym_table = s;
}

// Para entrar/salir de ámbitos
void enter_scope() { current_scope++; }
void exit_scope() {
    // Remover símbolos del ámbito actual
    Symbol *p = sym_table;
    while (p != NULL) {
        if (p->scope == current_scope) {
            // Remover (simplificado, en realidad necesita desenlazar)
            free(p->name);
            Symbol *temp = p;
            p = p->next;
            free(temp);
        } else {
            p = p->next;
        }
    }
    current_scope--;
}

// Catálogo de SKU simulado (para verificación de existencia)
const char *sku_catalog[] = {"camisa_M", "pantalon_azul", "zapato_42", NULL}; //
Aregar más según sea necesario
int is_sku(char *id) {
    for (int i = 0; sku_catalog[i] != NULL; i++) {

```

```
    if (strcmp(sku_catalog[i], id) == 0) return 1;
}
return 0;
}
```

Estas funciones permiten una gestión eficiente de la tabla de símbolos: lookup busca la declaración más reciente en el ámbito actual o superior, insert agrega nuevos símbolos con chequeo de redeclaración, y enter_scope/exit_scope manejan la visibilidad de variables locales. El catálogo de SKUs es un arreglo estático para simular datos externos, verificando existencia en comandos como VER_STOCK o REPONER.

A continuación se muestran las reglas principales con sus acciones semánticas integradas.

```
declaracion:
    VARIABLE IDENTIFICADOR ASIGNAR expresion
    {
        insert($2, $4.type, 0, T_UNKNOWN);
        printf("Declaración VARIABLE: %s de tipo %d\\n", $2, $4.type);
        free($2);
    }
    | CONSTANTE IDENTIFICADOR ASIGNAR expresion
    {
        insert($2, $4.type, 1, T_UNKNOWN);
        printf("Declaración CONSTANTE: %s de tipo %d\\n", $2, $4.type);
        free($2);
    }
    | ARREGLO IDENTIFICADOR ASIGNAR CORCHETE_IZQ lista_expresiones
    CORCHETE_DER
    {
        if ($5 == T_UNKNOWN) {
            yyerror("Error semantico: Arreglo vacio, tipo indefinido");
        }
        insert($2, T_ARRAY, 0, $5);
        printf("Declaración ARREGLO: %s de elementos tipo %d\\n", $2, $5);
        free($2);
    }
    ;
```

```
asignacion:
    IDENTIFICADOR ASIGNAR expresion
    {
        Symbol *s = lookup($1);
        if (s == NULL) {
            yyerror("Error semantico: Variable no declarada");
        }
    }
```

```

    } else if (s->is_const) {
        yyerror("Error semantico: Asignacion a constante");
    } else if (s->type != $3.type) {
        yyerror("Error semantico: Tipos incompatibles en asignacion");
    }
    printf("Asignación: %s = ...\n", $1);
    free($1);
}
;

```

reponer:

```

    REPONER IDENTIFICADOR expresion
    {
        if (!is_sku($2)) {
            yyerror("Error semantico: SKU no existe en catalogo");
        }
        if ($3.type != T_NUMBER || $3.num_val <= 0) {
            yyerror("Error semantico: Cantidad en REPONER debe ser numero positivo");
        }
        printf("Reponer producto %s con cantidad %.2f\n", $2, $3.num_val);
        free($2);
    }
;

```

aplicar_descuento:

```

    APLICAR_DESCUENTO IDENTIFICADOR NUMERO PORCENTAJE
    {
        if (!is_sku($2)) {
            yyerror("Error semantico: SKU no existe en catalogo");
        }
        if ($3 <= 0 || $3 > 100) {
            yyerror("Error semantico: Porcentaje en APLICAR_DESCUENTO debe estar
entre 0 y 100");
        }
        printf("Aplicar %.0f%% descuento a %s\n", $3, $2);
        free($2);
    }
    // (similar para versión condicional)
;

```

expresion:

```

    expresion MAS termino
    {
        if ($1.type != T_NUMBER || $3.type != T_NUMBER) {
            yyerror("Error semantico: Operandos en + deben ser numericos");
        }
        $$type = T_NUMBER;
        $$num_val = $1.num_val + $3.num_val;
        printf(" + ");
    }
;

```

```

    }
    // (otras operaciones similares con chequeo de tipos)
    ;

lista_expresiones:
    /* vacío */ { $$ = T_UNKNOWN; }
    | expresion { $$ = $1.type; }
    | lista_expresiones COMA expresion
      {
        if ($1 != $3.type) {
          yyerror("Error semantico: Elementos de arreglo con tipos mixtos");
        }
        $$ = $1;
      }
    ;

condicion:
    expresion IGUAL expresion
      {
        if ($1.type != $3.type) {
          yyerror("Error semantico: Tipos incompatibles en comparacion");
        }
        printf(" == ");
      }
    // (otras comparaciones con chequeo de tipos y numericos donde aplica)
    ;

si:
    SI condicion ENTONCES { enter_scope(); } programa FIN
      { printf("Estructura SI-ENTONCES-FIN\n"); exit_scope(); }
    // (similar para SINO)
    ;

para:
    PARA IDENTIFICADOR EN IDENTIFICADOR HAZ { enter_scope(); } programa
    FIN
      {
        Symbol *arr = lookup($4);
        if (arr == NULL || arr->type != T_ARRAY) {
          yyerror("Error semantico: Iterador EN debe ser un ARREGLO");
        }
        insert($2, arr->array_elem_type, 0, T_UNKNOWN);
        printf("Bucle PARA %s EN %s HAZ ... FIN\n", $2, $4);
        free($2); free($4);
        exit_scope();
      }
    ;

acceso_arreglo:

```

```

IDENTIFICADOR CORCHETE_IZQ expresion CORCHETE_DER
{
    Symbol *s = lookup($1);
    if (s == NULL || s->type != T_ARRAY) {
        yyerror("Error semantico: Acceso a no-arreglo");
    }
    if ($3.type != T_NUMBER || $3.num_val < 0) {
        yyerror("Error semantico: Indice de arreglo invalido");
    }
    $$type = s->array_elem_type;
    printf("Acceso a arreglo: %s[...]\n", $1);
    free($1);
}
;

```

4.2.5. Traducción Dirigido por la Sintaxis

En esta fase, el compilador de FashionOpsDSL evoluciona de ser solamente un analizador a un motor real de automatización de operaciones de e-commerce mediante una implementación con una base de datos.

A continuación se muestran unos ejemplos de cómo el DSL significa una ejecución de queries en la base de datos.

Construcción DSL	Acción generada
REPONER camisa_M 50;	UPDATE productos SET stock = stock + 50 WHERE sku = 'camisa_M';
APLICAR_DESCUENTO camisa_M 25%;	INSERT INTO descuentos (sku, porcentaje) VALUES ('camisa_M', 25.0);
VER_STOCK camisa_M ES cantidad;	SELECT stock FROM productos WHERE sku = 'camisa_M';

Para poder implementar la base de datos, se realizó cambios en el código ejecutado en el archivo bison para ejecutar las queries que realizarán cambios en la base de datos.

```

.
```

Dentro del main() es donde se inicia la conexión con la base de datos.

```


```

Una vez realizada la implementación del léxico, sintáctico y semántico se procede a usar los comandos flex, bison y gcc para generar un archivo ejecutable.

4.2.6. Aplicación Práctica

.

Conclusiones y Referencias

Referencias

- Debrouwere, S. (2025, 9 de mayo). Overcoming Key Challenges in Fashion and Apparel eCommerce. *Genie*.
<https://www.genie.io/blog-articles/overcoming-key-challenges-in-fashion-and-apparel-e-commerce>
- Jenkins, A. (s.f.). *Slow-Moving Inventory: Identify, Manage & Prevent It*. Oracle Netsuit.
Recuperado el 17 de octubre, 2025:
<https://www.netsuite.com/portal/resource/articles/inventory-management/slow-moving-inventory.shtml>
- JetBrains. (s.f.). *Lenguaje de dominio específico*. Recuperado el 16 de octubre, 2025:
<https://www.jetbrains.com/es-es/mps/concepts/domain-specific-languages/>
- Radial. (2024, 19 de agosto). *Tech takes on e-commerce's \$218 billion returns problem*.
Radial powering bpostgroup.
<https://www.radial.com/eur/insights/tech-takes-on-e-commerces-218-billion-returns-problem>
- Team, L. (2019, June 18). A domain specific language (DSL) for supply chain [Video].
<https://www.lokad.com/tv/2019/6/18/a-domain-specific-language>